

Big-O Notation

& JavaScript Memory Basics

■ What you'll learn today

- ◆ What Big-O notation means and why it matters
- ◆ Time complexity vs Space complexity
- ◆ Common complexity classes ($O(1) \rightarrow O(2^n)$)
- ◆ JavaScript memory: primitives vs references
- ◆ The 4 Golden Rules of Big-O
- ◆ Interview thinking framework

■ Study Plan (~1 Hour Total)

20 min

Theory

20 min

Code Examples

20 min

Practice Problems

Real-World Hook

Imagine searching for a book...

■ In a stack of 10 books → fast, easy

■■ In a library of 10,000,000 books → terrible if done wrong!

A bad algorithm works fine with 10 items... but collapses with millions.

What is Big-O?

$O(f(n))$

Describes how an algorithm's time or memory grows as input size n grows

■ NOT About

Exact milliseconds
Specific hardware speed

■ IS About

Growth behavior
Scalability as $n \rightarrow \infty$

Complexity Classes

NOTATION	NAME	PERFORMANCE
$O(1)$	Constant	★★★★★
$O(\log n)$	Logarithmic	★★★★■
$O(n)$	Linear	★★★██
$O(n \log n)$	Linearithmic	★★■■■
$O(n^2)$	Quadratic	★████
$O(2^n)$	Exponential	■■■■■

Example 1

Constant Time — $O(1)$

```
JavaScript

const arr = [10, 20, 30];

// Access element at index — always ONE step
console.log(arr[0]); //  $O(1)$ 
```

■ Even if arr has 3 or 3,000,000 items — accessing by index takes the SAME time.

Example 2

Linear Time — $O(n)$

```
JavaScript

const arr = [1, 2, 3, 4, 5];

// Loop visits every element once
for (let i = 0; i < arr.length; i++) {
  console.log(arr[i]); // n steps total
}
```

■ If the array doubles in size, the loop does twice the work → linear growth.

Example 3

Quadratic Time — $O(n^2)$

```
JavaScript

// NESTED loops → multiply the work
for (let i = 0; i < n; i++) {
  for (let j = 0; j < n; j++) {
    console.log(i, j); //  $n \times n = n^2$  steps
  }
}
```

■ $n=1000 \rightarrow 1,000,000$ operations! Avoid nested loops on large inputs.

The 4 Golden Rules of Big-O

1 Ignore Constants

$$O(5n) = O(n)$$

Constant multipliers vanish at scale.

2 Add Sequential Ops

$$O(n) + O(n) = O(n)$$

Two separate loops → still linear.

3 Multiply Nested Ops

$$O(n) \times O(n) = O(n^2)$$

Nested loops multiply complexity.

4 Keep Dominant Term

$$n^2 + n + 1 \rightarrow O(n^2)$$

Largest term dominates; drop the rest.

Space Complexity

Primitives

Number, Boolean, null, undefined

$$O(1)$$

Strings

Grows with string length

$$O(n)$$

Arrays/Objects

More items = more memory

$$O(n)$$

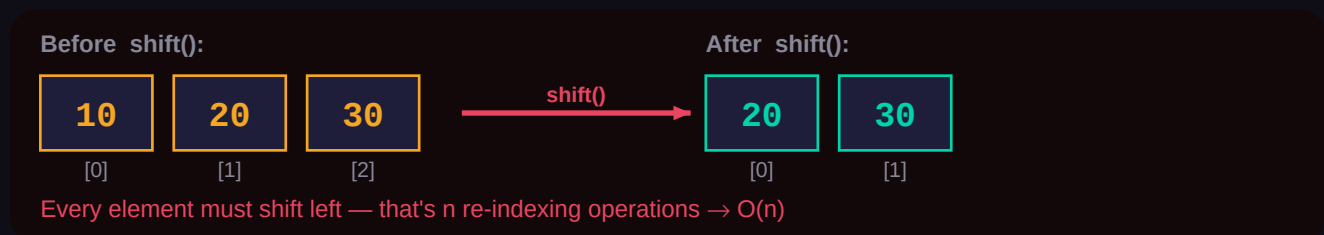
JavaScript Insight

Arrays in JS are specialized objects. This is why `push()` is $O(1)$ but `shift()` is $O(n)$.
`shift()` must re-index every element after removing index 0.

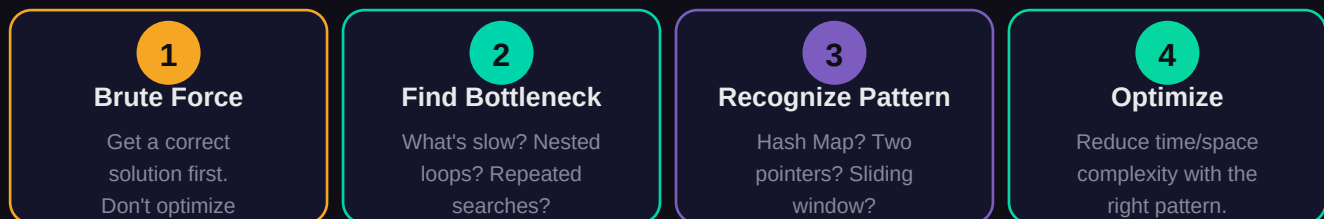
Array Operation Complexities

OPERATION	DESCRIPTION	COMPLEXITY
<code>push()</code>	Add to end	<code>O(1)</code>
<code>pop()</code>	Remove from end	<code>O(1)</code>
<code>index access</code>	<code>arr[i]</code>	<code>O(1)</code>
<code>shift()</code>	Remove from front	<code>O(n)</code>
<code>unshift()</code>	Add to front	<code>O(n)</code>

Why is `shift()` `O(n)`? — Visual



Interview Thinking Framework



Day 1 — Quick Reference Cheat Sheet

Array index access	$O(1)$	push() / pop()	$O(1)$
shift() / unshift()	$O(n)$	Linear search	$O(n)$
Nested loops	$O(n^2)$	Binary search	$O(\log n)$
Merge sort / Quick sort	$O(n \log n)$	String of length n	$O(n)$ space
Primitive variable	$O(1)$ space		

Practice Problems

■ Easy

Find Maximum Element

Hint: Loop through, track max variable

Reverse a String

Hint: Two-pointer swap from ends

Count Even Numbers

Hint: Loop + modulo check

■ Medium

Two Sum

Hint: Use a Hash Map for $O(n)$ lookup

Contains Duplicate

Hint: Hash Set: add & check exists

Valid Anagram

Hint: Sort both strings, compare

■ Common Beginner Mistakes

Memorizing without understanding • Ignoring space complexity • Using .shift() in large loops

■ Coming Next — Day 2

JavaScript Execution Context & Event Loop

Call stack, heap memory, event queue, microtasks vs macrotasks